Software Requirements Specification (SRS)

Uptoskills Intern Management & Workforce Tracking System

Version: 1.0
Prepared For: Uptoskills
Document Type: Enterprise Software Requirements Specification
Prepared By: System Architecture & Engineering Team

---

1. Introduction

1.1 Purpose of the System

The purpose of this system is to build a centralized, secure, scalable, and role-based Intern Management & Workforce Tracking Platform that can later be integrated into the Uptoskills ecosystem and portal.

The platform will help management efficiently manage:

- Interns
- Captains
- Team Leads (TLs)
- Senior Team Leads (Senior TLs)
- Administrative Operations
- Attendance Monitoring
- Performance Ratings
- Social Media Campaign Tracking
- Team Hierarchy Management
- Audit Logging
- Analytics & Reports
- Internal Workforce Operations

The system is designed to function similarly to corporate employee management systems used by modern organizations.

This platform will focus heavily on:

- Security
- Authorization
- Role hierarchy
- Accountability
- Tracking
- Scalability
- Maintainability

- Real-time monitoring

---

2. Vision of the Project

The vision of this project is to create a secure and enterprise-level workforce management platform specifically designed for internship and internal organizational management.

The system should:

- Eliminate manual tracking
- Reduce spreadsheet dependency
- Provide centralized control
- Improve accountability
- Provide role-based access
- Prevent unauthorized access
- Maintain structured hierarchy
- Allow future scaling
- Integrate with Uptoskills portal in future versions

This system should eventually behave like:

- Corporate HR systems
- Workforce management systems
- Internal productivity platforms
- Team hierarchy management systems

---

3. Scope of the Project

The system will include:

Core Functionalities

Authentication & Authorization

- Secure login system
- JWT authentication
- Refresh token handling
- Role-based access control (RBAC)
- Session tracking
- Protected APIs

Workforce Management

- Manage interns
- Manage captains
- Manage TLs
- Manage Senior TLs

- Manage departments
- Assign hierarchy

Attendance Management

- Daily attendance tracking
- Attendance history
- Monthly reports
- Attendance analytics
- Attendance verification

Rating & Evaluation System

- Performance rating
- Reviewer comments
- Historical performance tracking
- Analytics

Social Media Campaign Tracking

- Campaign/task creation
- Link distribution
- Screenshot proof upload
- Verification workflow
- Auto deletion after verification window

Analytics & Reporting

- Dashboard analytics
- Attendance insights
- Performance insights
- Team productivity reports

Audit & Monitoring

- Activity tracking
- Login monitoring
- Permission tracking
- Sensitive action logging

---

4. Business Objectives

The main business objectives are:

1. Centralize management operations
2. Improve tracking of interns and teams
3. Create accountability across hierarchy
4. Ensure secure access control
5. Simplify attendance tracking
6. Improve management transparency

7. Reduce manual workload
8. Create scalable internal infrastructure
9. Build reusable organizational architecture
10. Create future-ready integration system for Uptoskills

---

5. User Roles & Hierarchy

The platform follows strict hierarchical authorization.

5.1 Role Hierarchy

ADMIN ↓ SENIOR TL ↓ TL ↓ CAPTAIN ↓ INTERN

---

6. Detailed Role Permissions

6.1 ADMIN

The ADMIN role has complete system-level access.

Admin Permissions

- Access all users
- Access all attendance records
- Access all ratings
- Manage Senior TLs
- Assign roles
- Delete/suspend accounts
- View complete analytics
- Access all uploaded proofs
- Access audit logs
- Access all departments
- Manage campaigns
- Manage notifications
- View all reports
- System-wide management

Restrictions

None.

6.2 SENIOR TL

Senior TL manages TLs and Captains under assigned departments.

Permissions

- Access assigned TLs
- Access assigned Captains
- Access assigned interns
- Manage TL attendance
- Manage Captain attendance
- Assign ratings to TLs
- Assign ratings to Captains
- Verify social proof submissions
- View analytics of assigned departments
- Access reports

Restrictions

- Cannot access Admin routes
- Cannot manage unrelated departments
- Cannot access global system configuration

6.3 TL (Team Lead)

Permissions

- Access assigned captains
- Access assigned interns
- View attendance
- Manage attendance of assigned members
- Rate captains
- Verify social tasks
- View analytics of assigned teams

Restrictions

- Cannot access Senior TL management
- Cannot access Admin data
- Cannot access unrelated teams

6.4 CAPTAIN

Permissions

- Manage interns under them
- Mark attendance
- Rate interns
- Verify intern social submissions
- View intern reports

Restrictions

- Cannot access TL or Senior TL data
- Cannot access unrelated interns

6.5 INTERN

Permissions

- Login to own dashboard
- View own attendance
- View own ratings
- Upload screenshot proofs
- View assigned tasks

Restrictions

- Cannot access other users
- Cannot access management data
- Cannot modify attendance
- Cannot modify ratings

---

7. Authentication System

7.1 Authentication Type

The system will use:

- Email-based authentication
- JWT access tokens
- Refresh token rotation
- Role-based authorization

7.2 Login Process

Login Flow

1. User enters registered email and password
2. Credentials verified
3. Password validated using Argon2
4. JWT access token generated
5. Refresh token generated
6. Secure session created
7. User redirected to role-specific dashboard

---

8. Authorization Architecture

Authorization is the most important component of the system.

The system will use:

- RBAC (Role-Based Access Control)
- Ownership-based validation
- Route-level protection
- Middleware authorization
- API-level restrictions

---

9. Security Architecture

The system will follow enterprise-level security standards.

9.1 Security Features

Authentication Security

- JWT Authentication
- Refresh token rotation
- Secure password hashing using Argon2
- Session tracking
- Device tracking

API Security

- Protected APIs
- Middleware validation
- Request validation
- Zod schema validation
- Centralized error handling

Infrastructure Security

- Rate limiting
- Brute-force protection
- Secure headers
- CORS protection
- CSRF protection
- SQL injection prevention
- XSS mitigation

File Upload Security

- MIME validation
- File size validation

- Secure upload storage
- Unique file naming
- Auto deletion

Logging Security

- Audit logs
- Login activity logs
- Failed login tracking
- Sensitive operation logs

---

10. Social Media Task Module

10.1 Purpose

This module will help management track social media engagement activities.

10.2 Workflow

Step 1

Admin or Senior TL creates a social media task.

Example:

- LinkedIn repost
- Instagram story
- Comment campaign
- Share campaign

Step 2

Task link distributed to users.

Step 3

Intern performs task.

Step 4

Intern uploads screenshot proof.

Step 5

Captain/TL/Senior TL verifies submission.

Step 6

Proof remains accessible for 24 hours.

Step 7

Proof image auto-deletes after 24 hours.

Step 8

Verification data remains permanently stored.

---

11. Attendance Management System

11.1 Attendance Features

- Daily attendance
- Present/Absent/Late status
- Attendance remarks
- Monthly reports
- Attendance analytics
- Attendance history
- Role-wise filtering

11.2 Attendance Hierarchy

Captain

Can mark intern attendance.

TL

Can manage captain and intern attendance.

Senior TL

Can manage TL and captain attendance.

Admin

Can manage attendance of all users.

---

12. Rating & Performance System

12.1 Rating Features

- Monthly ratings
- Performance evaluation
- Reviewer comments
- Rating history
- Analytics
- Team performance tracking

12.2 Rating Hierarchy

Captain → Intern

TL → Captain

Senior TL → TL + Captain

Admin → Senior TL + All Roles

---

13. Dashboard System

Each role will have its own dashboard.

13.1 Admin Dashboard

Features:

- Global analytics
- User management
- Attendance overview
- Ratings overview
- Department overview
- Security monitoring
- Audit logs
- Reports

13.2 Senior TL Dashboard

Features:

- Team analytics

- TL management
- Captain management
- Attendance overview
- Ratings overview

13.3 TL Dashboard

Features:

- Captain management
- Intern tracking
- Team attendance
- Team analytics

13.4 Captain Dashboard

Features:

- Intern management
- Attendance marking
- Ratings
- Proof verification

13.5 Intern Dashboard

Features:

- View tasks
- Upload proofs
- View attendance
- View ratings

---

14. Notifications System

The platform will support notifications.

Notification Examples

- New task assigned
- Attendance missing
- Rating updated
- Proof approved
- Proof rejected

- System announcements

---

15. Audit Logging System

The system will maintain enterprise-level audit logs.

Activities Tracked

- Login activity
- Failed login attempts
- Attendance updates
- Rating updates
- Role changes
- Task verification
- Proof uploads
- Sensitive actions

---

16. Database Design

Main Database

PostgreSQL

Main Tables

users

Stores all user data.

roles

Stores role definitions.

departments

Stores department information.

attendance

Stores attendance records.

ratings

Stores performance ratings.

social_tasks

Stores social campaigns.

proof_submissions

Stores uploaded proof metadata.

notifications

Stores notifications.

audit_logs

Stores system logs.

sessions

Stores active sessions.

---

17. Database Standards

The database will use:

- UUID primary keys
- Foreign keys
- Indexing
- Constraints
- Transactions
- Soft deletes
- Timestamp tracking
- Optimized queries

---

18. Complete Technology Stack & Engineering Architecture

This section defines the exact technologies, frameworks, architecture patterns, security layers, development flow, infrastructure flow, database flow, API architecture, deployment structure, and complete engineering ecosystem of the platform.

18.1 Frontend Technology Stack

The frontend will be lightweight, scalable, responsive, and role-based.

Frontend Framework

- Next.js
- React
- TypeScript

UI & Styling

- Tailwind CSS
- Shadcn UI
- Framer Motion (minimal use)

State Management

- Zustand or Context API

API Handling

- Axios
- React Query / TanStack Query

Form Handling

- React Hook Form
- Zod Validation

Frontend Security

- Route protection
- Role-based rendering
- Protected dashboard layouts
- Secure token handling
- CSRF-safe API calls

18.2 Backend Technology Stack

The backend is the core of the system.

The platform will use a modular monolithic enterprise backend architecture.

Backend Runtime

- Node.js

Backend Language

- TypeScript

Backend Framework

- Fastify

Why Fastify?

- Faster than Express
- Better scalability
- Better TypeScript support
- Lightweight
- Production efficient

Database Driver

- pg (PostgreSQL Driver)

Database

- PostgreSQL

Cache & Session Layer

- Redis

Validation

- Zod

Logging

- Pino Logger

Password Hashing

- Argon2

Authentication

- JWT Access Token
- Refresh Token Rotation

File Upload Handling

- Multer / Fastify Multipart

Upload Storage

- Cloudinary / S3 compatible storage

Background Jobs

- Node Cron

API Documentation

- Swagger / OpenAPI

18.3 Infrastructure Architecture

The platform architecture is divided into multiple logical layers.

Client Layer ↓ Frontend Layer (Next.js) ↓ API Gateway Layer (Fastify APIs) ↓ Service Layer ↓ Authorization Layer ↓ Database Layer (PostgreSQL) ↓ Cache Layer (Redis) ↓ Storage Layer (Cloudinary/S3)

18.4 Authentication Architecture

Authentication will follow enterprise-level standards.

Authentication Flow

User Login ↓ Credentials Validation ↓ Password Verification (Argon2) ↓ JWT Access Token Generation ↓ Refresh Token Generation ↓ Session Storage in Redis ↓ Role Validation ↓ Dashboard Access

---

18.5 Authorization Architecture

Authorization is the most critical part of the platform.

The platform uses:

- RBAC (Role-Based Access Control)
- Ownership Validation
- Hierarchy Validation
- Route Guards
- Middleware Authorization

Example Authorization Rules

Intern

Cannot access:

- Captain APIs
- TL APIs
- Senior TL APIs
- Admin APIs

Captain

Can only access:

- Assigned interns

Cannot access:

- Other captain interns
- TL dashboards

TL

Can access:

- Captains under them
- Interns under assigned captains

Senior TL

Can access:

- TLs under assigned departments
- Captains
- Interns

Admin

Can access entire system.

18.6 Security Architecture

The platform will implement production-grade security standards.

API Security

- JWT verification
- Refresh token rotation

- Route guards
- Role middleware
- Ownership validation
- Request validation
- Input sanitization
- Centralized error handling

Infrastructure Security

- Rate limiting
- Brute-force protection
- CSRF protection
- CORS protection
- Secure headers
- XSS prevention
- SQL injection prevention

File Upload Security

- MIME validation
- File size validation
- Secure upload storage
- Auto deletion
- Duplicate prevention

Logging Security

- Audit logs
- Login tracking
- Failed login tracking
- Sensitive operation tracking

18.7 Database Architecture

Database Type

Relational Database

Database Used

PostgreSQL

Database Design Principles

- UUID Primary Keys
- Foreign Key Constraints
- Normalized Schema

- Indexed Queries
- Transaction Safety
- Soft Deletes
- Timestamp Tracking

18.8 Database Tables Structure

users

Stores:

- user profile
- role
- department
- email
- password
- hierarchy mapping

roles

Stores role definitions.

departments

Stores departments.

attendance

Stores attendance records.

ratings

Stores rating records.

social_tasks

Stores campaigns/tasks.

proof_submissions

Stores uploaded proof metadata.

audit_logs

Stores audit trails.

notifications

Stores notifications.

sessions

Stores active sessions.

18.9 Complete System Flow

Admin Flow

Admin Login ↓ Access Global Dashboard ↓ Manage Departments ↓ Manage Senior TLs ↓ View Analytics ↓ Access Reports ↓ Manage System

---

Senior TL Flow

Senior TL Login ↓ View Assigned Departments ↓ Manage TLs ↓ Manage Captains ↓ Verify Activities ↓ Monitor Performance

---

TL Flow

TL Login ↓ Manage Captains ↓ Track Interns ↓ Monitor Attendance ↓ Review Performance

---

Captain Flow

Captain Login ↓ Access Intern Dashboard ↓ Mark Attendance ↓ Verify Proof Uploads ↓ Rate Interns

---

Intern Flow

Intern Login ↓ Access Dashboard ↓ View Tasks ↓ Upload Proof ↓ Track Attendance & Ratings

---

18.10 Proof Verification Flow

Admin/Senior TL Creates Task ↓ Task Assigned to Interns ↓ Intern Uploads Screenshot Proof ↓ Captain/TL/Senior TL Verifies ↓ Verification Status Saved ↓ Image Auto Deletes After 24 Hours ↓ Verification Record Remains Permanently

---

18.11 Attendance System Flow

Captain/TL/Senior TL Marks Attendance ↓ Attendance Stored in PostgreSQL ↓ Analytics Updated ↓ Reports Generated ↓ Dashboard Updated

---

18.12 Ratings Flow

Reviewer Assigns Rating ↓ Rating Saved ↓ Analytics Updated ↓ Historical Records Maintained

---

18.13 Backend Folder Structure

backend/ ├── src/ │ ├── modules/ │ ├── auth/ │ ├── users/ │ ├── attendance/ │ ├── ratings/ │ ├── uploads/ │ ├── notifications/ │ ├── audit/ │ ├── middleware/ │ ├── validators/ │ ├── services/ │ ├── repositories/ │ ├── db/ │ ├── routes/ │ ├── utils/ │ ├── config/ │ ├── types/ │ └── logs/

18.14 Frontend Folder Structure

frontend/ ├── app/ ├── dashboard/ ├── auth/ ├── components/ ├── layouts/ ├── services/ ├── hooks/ ├── api/ ├── store/ ├── utils/ └── types/

18.15 API Architecture

The backend will expose:

- REST APIs
- Secure APIs
- Role-protected APIs
- Versioned APIs

Example:

/api/v1/auth/login /api/v1/attendance /api/v1/ratings /api/v1/tasks /api/v1/users

---

18.16 API Request Lifecycle

Frontend Request ↓ Authentication Middleware ↓ Authorization Middleware ↓ Validation Layer ↓ Controller Layer ↓ Service Layer ↓ Repository Layer ↓ PostgreSQL ↓ Response Returned

---

18.17 Redis Architecture

Redis will be used for:

- Session tracking
- Refresh token storage
- Rate limiting
- Caching
- Temporary security data

18.18 Development Workflow

Phase 1

Backend Core

- Authentication
- Authorization
- Database
- Attendance APIs
- Ratings APIs

Phase 2

Frontend Development

- Dashboards
- Role layouts
- Upload flows
- Analytics pages

Phase 3

Security Hardening

- Rate limiting
- Audit logs
- Upload security
- Session protection

Phase 4

Testing & Optimization

- Bug fixing
- API optimization
- Query optimization

- Security testing

18.19 Deployment Architecture

Frontend ↓ Vercel / VPS

Backend ↓ Linux VPS / Cloud Server

Database ↓ PostgreSQL Server

Redis ↓ Redis Instance

Storage ↓ Cloudinary / S3

18.20 Future Scalability

Future planned scalability:

- Google OAuth
- Mobile App
- AI Analytics
- Real-time Notifications
- Multi-tenant architecture
- Department scaling
- HR integration
- Advanced reporting

---

19. Recommended Architecture

Backend

- Node.js
- TypeScript
- Fastify
- PostgreSQL
- pg Driver
- Redis
- JWT
- Argon2
- Zod
- Pino Logger

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- Shadcn UI

---

19. Recommended Architecture

Architecture Type

Modular Monolith Architecture

Why Modular Monolith?

- Easier development
- Better maintainability
- Easier debugging
- Faster delivery
- Cleaner structure
- Scalable enough for future

---

20. Backend Architecture

Folder Structure

backend/ ├── modules/ ├── auth/ ├── users/ ├── attendance/ ├── ratings/ ├── uploads/ ├── notifications/ ├── audit/ ├── middleware/ ├── db/ ├── services/ ├── validators/ ├── utils/ ├── config/ ├── routes/ ├── types/ └── logs/

---

21. Frontend Architecture

frontend/ ├── app/ ├── dashboard/ ├── auth/ ├── components/ ├── layouts/ ├── api/ ├── hooks/ ├── store/ ├── services/ ├── utils/ └── types/

---

22. API Architecture

The backend will expose:

- REST APIs
- Versioned APIs
- Secure APIs
- Role-protected APIs

---

23. API Security Standards

All APIs must:

- Validate JWT
- Validate roles
- Validate ownership
- Use Zod validation
- Use centralized error handling
- Use proper HTTP status codes
- Prevent injection attacks

---

24. File Upload Architecture

Upload Flow

1. User uploads screenshot
2. Backend validates file
3. File stored securely
4. Metadata stored in PostgreSQL
5. Verification status updated
6. Cron job deletes file after 24h

---

25. Cron Jobs & Automation

The system will include background jobs.

Jobs

- Auto delete screenshots
- Send notifications
- Generate attendance reports
- Session cleanup
- Token cleanup

26. Analytics System

Analytics will include:

- Attendance percentage
- Team performance
- Active interns
- Inactive interns
- Rating trends
- Department performance
- Pending approvals

---

27. Scalability Plan

Future scalability support:

- Multi-department support
- Cloud storage migration
- Google OAuth integration
- Email service integration
- Real-time notifications
- AI analytics
- Mobile application integration

---

28. Integration with Uptoskills Portal

The system is designed to later integrate with Uptoskills.

Future integration areas:

- Shared authentication
- Shared user database
- Shared analytics
- Shared dashboard systems
- Unified management

---

29. Development Phases

Phase 1

Core Backend:

- Authentication
- Authorization
- Database
- Attendance APIs
- Ratings APIs

Phase 2

Frontend:

- Dashboards
- Attendance UI
- Upload UI
- Analytics UI

Phase 3

Security Hardening:

- Rate limiting
- Audit logs
- CSRF
- Upload validation

Phase 4

Optimization & Deployment:

- Performance optimization
- Bug fixing
- Testing
- Deployment

---

30. Team Structure Recommendation

Backend Team

- Authentication
- APIs
- Database
- Security

Frontend Team

- Dashboards
- Components
- UI

QA Team

- Testing
- Bug reports
- Validation

DevOps Team

- Deployment
- Infrastructure
- Monitoring

---

31. Estimated Timeline

MVP Version

10–14 days

Production Version

3–5 weeks

Full Enterprise Version

2–3 months

---

32. Final System Goals

The final platform should:

- Be secure
- Be scalable
- Be maintainable
- Be enterprise-ready
- Be integration-ready
- Be role-protected
- Be audit-ready
- Be analytics-driven
- Be production-grade

---

33. Most Important Engineering Goals

The most critical engineering areas are:

1. Authorization Architecture
2. Role Hierarchy
3. API Security
4. Attendance Integrity
5. Database Design
6. Audit Logging
7. Session Security
8. Ownership Validation

---

34. Conclusion

The Uptoskills Intern Management & Workforce Tracking System is designed as a modern enterprise-grade management platform focused on secure hierarchy management, attendance tracking, workforce analytics, role-based authorization, and operational scalability.

The system architecture is intentionally designed to support future growth, future integration with the Uptoskills ecosystem, and long-term organizational management needs.

This project is not just an internship tracker. It is a scalable workforce operations platform built using modern engineering practices, enterprise security principles, and modular architecture standards.